1

```
module D_array();

    bit [11:0] my_array [4];

    initial begin
        my_array = '{12'h012, 12'h345, 12'h678, 12'h9AB};

        for (int i = 0; i < $size(my_array); i++) begin
            $display("Bits[5:4] of %b is %b",my_array[i],my_array[i][5:4]);
        end

        $display("----------------------------------");
        foreach(my_array[i]) begin
            $display("Bits[5:4] of %b is %b",my_array[i],my_array[i][5:4]);
        end
    end

endmodule
```

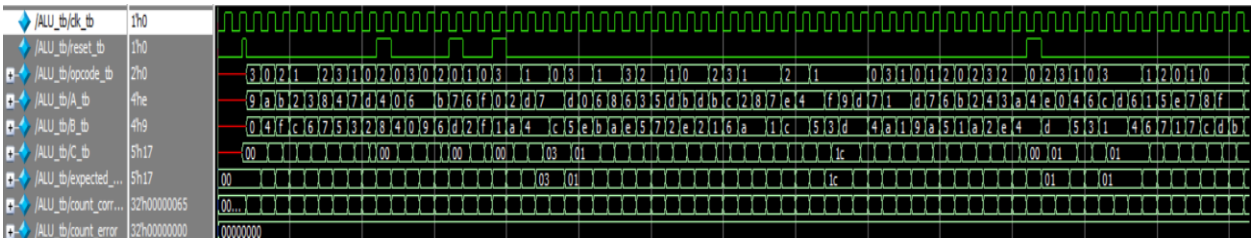
```
# run -all
# Bits[5:4] of 000000010010 is 01
# Bits[5:4] of 001101000101 is 00
# Bits[5:4] of 011001111000 is 11
# Bits[5:4] of 100110101011 is 10
# ----------------------------------
# Bits[5:4] of 000000010010 is 01
# Bits[5:4] of 001101000101 is 00
# Bits[5:4] of 011001111000 is 11
# Bits[5:4] of 100110101011 is 10
```

```
package ALU_pkg;

    typedef enum bit [2:0] {ADD, SUB, NOT_A, ReductionOR_B} opcode_e;

    class ALU_cls;
        rand bit reset;
        rand reg signed [3:0] A,B;
        rand opcode_e opcode;

        constraint rst_cst{
            reset dist {0:= 95, 1:= 5};
        }
    endclass

endpackage
```

```
import ALU_pkg::*;

module ALU_tb;

    // Declare local wire and reg
    reg clk_tb;
    reg reset_tb;
    reg [1:0] opcode_tb;
    reg signed [3:0] A_tb;
    reg signed [3:0] B_tb;
    wire signed [4:0] C_tb;

    bit signed [4:0] expected_result;
    // Instantiate the module under test
    ALU_4_bit uut(
        clk_tb,
        reset_tb,
        opcode_tb,
        A_tb,
        B_tb,
        C_tb
    );

    // define local parameter for tracking the functionality
    integer  count_correct, count_error;

   initial begin
    clk_tb = 1'b0;
    forever #5 clk_tb = ~clk_tb;
   end

    // create object of class
    ALU_cls ALU_obj = new();
```

```
// golden model
task golden_model (input ALU_cls ALU_obj1);
    if(ALU_obj1.reset) begin
        expected_result = 5'b0;
    end else begin
        case (ALU_obj1.opcode)
            ADD:            expected_result = A_tb + B_tb;
            SUB:            expected_result = A_tb - B_tb;
            NOT_A:          expected_result = ~A_tb;
            ReductionOR_B:  expected_result = |B_tb;
            default:  expected_result = 5'b0;
        endcase
    end

endtask

// define task for checking
task checking(input signed [4:0] expected_result);
    @(negedge clk_tb);
    if (expected_result !== C_tb) begin
        count_error += 1;
        $display("A = %d, B = %d, expected = %d, Actual = %d",A_tb,B_tb,expected_result,C_tb);
    end
    else begin
        count_correct += 1;
    end
endtask

// define task for reset functionality
task assert_reset;
    reset_tb = 1'b1;
    checking(0);
    reset_tb = 1'b0;
endtask
```

```
    initial begin

        count_correct = 0;
        count_error = 0;
        reset_tb = 1'b0;
        #17;

        assert_reset;
        repeat(100) begin
            ALU_obj.randomize();
            reset_tb = ALU_obj.reset;
            A_tb = ALU_obj.A;
            B_tb = ALU_obj.B;
            opcode_tb = ALU_obj.opcode;
            golden_model(ALU_obj);
            checking(expected_result);
        end
        $display("Total correct cases = %d", count_correct);
        $display("Total error cases = %d", count_error);
        $stop;
    end
endmodule
```




```
================================================================================
=== Instance: /ALU_tb/uut
=== Design Unit: work.ALU_4_bit
================================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                         6         6         0   100.00%

=================================Branch Details=================================
```

```
Statement Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Statements                       8         8         0   100.00%

================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         44        44         0   100.00%

================================Toggle Details=================================
```

3

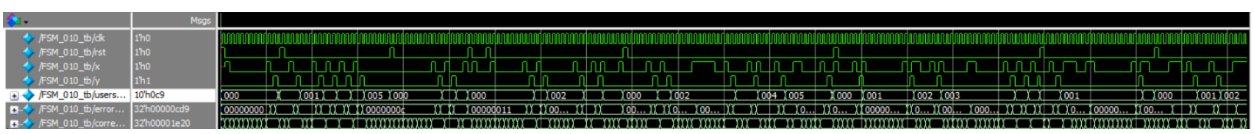
```
package FSM_pkg;

    typedef enum bit [1:0] {IDLE, ZERO, ONE, STORE} state_e;

    class fsm_transaction;
        rand bit rst, x;
        bit y_exp;
        bit [9:0] users_count_exp;

        constraint rst_cst{
            rst dist {0:= 97, 1:=3};
        }

        constraint x_cst{
            x dist {0:= 67, 1:=33};
        }

    endclass

endpackage
```

```
import FSM_pkg::*;

module FSM_010_tb();

    //1- Delcare local wir and reg
    logic clk, rst, x;
    logic y;
    logic [9:0] users_count;

    fsm_transaction fsm_obj = new();

    integer error_count, correct_count;

    //2- Instantiate the module under test
    FSM_010 uut(
        clk,
        rst,
        x,
        y,
        users_count);

    //3- Generate clock
    initial begin
        clk = 1'b0;
        forever #5 clk = ~clk;
    end
```

```
//4- Stimulus generation
initial begin
    error_count = 0;
    correct_count = 0;

    // Assert reset
    assert_reset();

    // Generate random transactions
    repeat (1000) begin
        assert(fsm_obj.randomize());
        rst = fsm_obj.rst;
        x = fsm_obj.x;
        check_result(fsm_obj);
    end

    // Generate random transactions
    repeat (10000) begin
        assert(fsm_obj.randomize());
        fsm_obj.rst = 0; // Ensure reset is deasserted for this part of the test
        rst = 0;
        x = fsm_obj.x;
        check_result(fsm_obj);
    end

    $display("Total tests: %d, Passed: %d, Failed: %d", error_count + correct_count, correct_count, error_count);
    $stop;
end
```

```
task golden_model(input fsm_transaction fsm_obj1);
    state_e cs, ns;
    case (cs)
        IDLE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = ZERO;
        ZERO:
            if (fsm_obj.x)
                ns = ONE;
            else
                ns = ZERO;
        ONE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = STORE;
        STORE:
            if (fsm_obj.x)
                ns = IDLE;
            else
                ns = ZERO;
        default:    ns = IDLE;
    endcase

    @(posedge clk);
        if(fsm_obj.rst) begin
            cs = IDLE;
            fsm_obj.users_count_exp = 0;
        end
        else begin
            cs = ns;
            fsm_obj.users_count_exp =  (cs == STORE) ? fsm_obj.users_count_exp + 1 : fsm_obj.users_count_exp;
        end
    fsm_obj.y_exp = (cs == ONE) ? 1'b1 : 1'b0;
endtask
```

```
    task assert_reset;
        rst = 1'b1;
        check_reset();
        rst = 1'b0;
    endtask

    task check_result(input fsm_transaction fsm_obj2);
        golden_model(fsm_obj2);
        if (!fsm_obj2.rst) begin
            @(negedge clk);
            if ((fsm_obj2.y_exp !== y) || (fsm_obj2.users_count_exp !== users_count)) begin
                error_count += 1;
                $display("Test failed for y.Expected: %b, Y: %b", fsm_obj2.y_exp, y);
            end else begin
                correct_count += 1;
                $display("Test passed for y.Expected: %b, Y: %b", fsm_obj2.y_exp, y);
            end
        end else begin
            check_reset();
        end
    endtask;

    task check_reset;
         @(negedge clk);
            if ((y !== 0) || (users_count !== 0)) begin
                error_count += 1;
                $display("Reset test failed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
            end else begin
                correct_count += 1;
                $display("Reset test passed. Expected y: 0, y: %b, Expected users_count: 0, users_count: %d", y, users_count);
            end
    endtask


endmodule
```




```
==============================================================================
=== Instance: /FSM_010_tb/uut
=== Design Unit: work.FSM_010
==============================================================================
Branch Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Branches                        20        20         0   100.00%

================================Branch Details================================
```

```
Condition Coverage:
    Enabled Coverage              Bins    Covered    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Conditions                       2         2         0   100.00%

================================Condition Details================================
```

```
FSM Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    FSM States                       4         4         0   100.00%
    FSM Transitions                  7         7         0   100.00%

================================FSM Details================================
```

```
 Statement Coverage:
     Enabled Coverage              Bins      Hits    Misses  Coverage
     ----------------              ----      ----    ------  --------
     Statements                      16        16         0   100.00%

 ================================Statement Details================================
```

```
Toggle Coverage:
    Enabled Coverage              Bins      Hits    Misses  Coverage
    ----------------              ----      ----    ------  --------
    Toggles                         36        36         0   100.00%

================================Toggle Details================================
```